
Unsupervised Domain Adaptation by Backpropagation

Yaroslav Ganin
Victor Lempitsky

Skolkovo Institute of Science and Technology (Skoltech)

GANIN@SKOLTECH.RU
LEMPITSKY@SKOLTECH.RU

Abstract

Top-performing deep architectures are trained on massive amounts of labeled data. In the absence of labeled data for a certain task, domain adaptation often provides an attractive option given that labeled data of similar nature but from a different domain (e.g. synthetic images) are available. Here, we propose a new approach to domain adaptation in deep architectures that can be trained on large amount of labeled data from the source domain and large amount of unlabeled data from the target domain (no labeled target-domain data is necessary).

As the training progresses, the approach promotes the emergence of “deep” features that are (i) discriminative for the main learning task on the source domain and (ii) invariant with respect to the shift between the domains. We show that this adaptation behaviour can be achieved in almost any feed-forward model by augmenting it with few standard layers and a simple new *gradient reversal* layer. The resulting augmented architecture can be trained using standard backpropagation.

Overall, the approach can be implemented with little effort using any of the deep-learning packages. The method performs very well in a series of image classification experiments, achieving adaptation effect in the presence of big domain shifts and outperforming previous state-of-the-art on Office datasets.

1. Introduction

Deep feed-forward architectures have brought impressive advances to the state-of-the-art across a wide variety of machine-learning tasks and applications. At the moment, however, these leaps in performance come only when a large amount of labeled training data is available. At the same time, for problems lacking labeled data, it may be still possible to obtain training sets that are big enough for training large-scale deep models, but that suffer from the *shift* in data distribution from the actual data encountered

at “test time”. One particularly important example is synthetic or semi-synthetic training data, which may come in abundance and be fully labeled, but which inevitably have a distribution that is different from real data (Liebelt & Schmid, 2010; Stark et al., 2010; Vázquez et al., 2014; Sun & Saenko, 2014).

Learning a discriminative classifier or other predictor in the presence of a *shift* between training and test distributions is known as *domain adaptation* (DA). A number of approaches to domain adaptation has been suggested in the context of *shallow* learning, e.g. in the situation when data representation/features are given and fixed. The proposed approaches then build the mappings between the *source* (training-time) and the *target* (test-time) domains, so that the classifier learned for the source domain can also be applied to the target domain, when composed with the learned mapping between domains. The appeal of the domain adaptation approaches is the ability to learn a mapping between domains in the situation when the target domain data are either fully unlabeled (*unsupervised domain annotation*) or have few labeled samples (*semi-supervised domain adaptation*). Below, we focus on the harder unsupervised case, although the proposed approach can be generalized to the semi-supervised case rather straightforwardly.

Unlike most previous papers on domain adaptation that worked with fixed feature representations, we focus on combining domain adaptation and deep feature learning within one training process (*deep domain adaptation*). Our goal is to embed domain adaptation into the process of learning representation, so that the final classification decisions are made based on features that are both discriminative and invariant to the change of domains, i.e. have the same or very similar distributions in the source and the target domains. In this way, the obtained feed-forward network can be applicable to the target domain without being hindered by the shift between the two domains.

We thus focus on learning features that combine (i) discriminativeness and (ii) domain-invariance. This is achieved by jointly optimizing the underlying features as well as two discriminative classifiers operating on these features: (i) the *label predictor* that predicts class labels and is used both during training and at test time and (ii) the

domain classifier that discriminates between the source and the target domains during training. While the parameters of the classifiers are optimized in order to minimize their error on the training set, the parameters of the underlying deep feature mapping are optimized in order to *minimize* the loss of the label classifier and to *maximize* the loss of the domain classifier. The latter encourages domain-invariant features to emerge in the course of the optimization.

Crucially, we show that all three training processes can be embedded into an appropriately composed deep feed-forward network (Figure 1) that uses standard layers and loss functions, and can be trained using standard backpropagation algorithms based on stochastic gradient descent or its modifications (e.g. SGD with momentum). Our approach is generic as it can be used to add domain adaptation to any existing feed-forward architecture that is trainable by backpropagation. In practice, the only non-standard component of the proposed architecture is a rather trivial *gradient reversal* layer that leaves the input unchanged during forward propagation and reverses the gradient by multiplying it by a negative scalar during the backpropagation.

Below, we detail the proposed approach to domain adaptation in deep architectures, and present results on traditional deep learning image datasets (such as MNIST (LeCun et al., 1998) and SVHN (Netzer et al., 2011)) as well as on OFFICE benchmarks (Saenko et al., 2010), where the proposed method considerably improves over previous state-of-the-art accuracy.

2. Related work

A large number of domain adaptation methods have been proposed over the recent years, and here we focus on the most related ones. Multiple methods perform unsupervised domain adaptation by matching the feature distributions in the source and the target domains. Some approaches perform this by reweighing or selecting samples from the source domain (Borgwardt et al., 2006; Huang et al., 2006; Gong et al., 2013), while others seek an explicit feature space transformation that would map source distribution into the target ones (Pan et al., 2011; Gopalan et al., 2011; Baktashmotlagh et al., 2013). An important aspect of the distribution matching approach is the way the (dis)similarity between distributions is measured. Here, one popular choice is matching the distribution means in the kernel-reproducing Hilbert space (Borgwardt et al., 2006; Huang et al., 2006), whereas (Gong et al., 2012; Fernando et al., 2013) map the principal axes associated with each of the distributions. Our approach also attempts to match feature space distributions, however this is accomplished by modifying the feature representation itself rather than by reweighing or geometric transformation. Also, our method uses (implicitly) a rather different way to measure the disparity between distributions based on their separability by a deep discriminatively-trained classifier.

Several approaches perform gradual transition from the source to the target domain (Gopalan et al., 2011; Gong et al., 2012) by a gradual change of the training distribution. Among these methods, (S. Chopra & Gopalan, 2013) does this in a “deep” way by the layerwise training of a sequence of deep autoencoders, while gradually replacing source-domain samples with target-domain samples. This improves over a similar approach of (Glorot et al., 2011) that simply trains a single deep autoencoder for both domains. In both approaches, the actual classifier/predictor is learned in a separate step using the feature representation learned by autoencoder(s). In contrast to (Glorot et al., 2011; S. Chopra & Gopalan, 2013), our approach performs feature learning, domain adaptation and classifier learning jointly, in a unified architecture, and using a single learning algorithm (backpropagation). We therefore argue that our approach is simpler (both conceptually and in terms of its implementation). Our method also achieves considerably better results on the popular OFFICE benchmark.

While the above approaches perform unsupervised domain adaptation, there are approaches that perform *supervised* domain adaptation by exploiting labeled data from the target domain. In the context of deep feed-forward architectures, such data can be used to “fine-tune” the network trained on the source domain (Zeiler & Fergus, 2013; Oquab et al., 2014; Babenko et al., 2014). Our approach does not require labeled target-domain data. At the same time, it can easily incorporate such data when it is available.

An idea related to ours is described in (Goodfellow et al., 2014). While their goal is quite different (building generative deep networks that can synthesize samples), the way they measure and minimize the discrepancy between the distribution of the training data and the distribution of the synthesized data is very similar to the way our architecture measures and minimizes the discrepancy between feature distributions for the two domains.

Finally, a recent and concurrent report by (Tzeng et al., 2014) also focuses on domain adaptation in feed-forward networks. Their set of techniques measures and minimizes the distance of the data means across domains. This approach may be regarded as a “first-order” approximation to our approach, which seeks a tighter alignment between distributions.

3. Deep Domain Adaptation

3.1. The model

We now detail the proposed model for the domain adaptation. We assume that the model works with input samples $\mathbf{x} \in X$, where X is some input space and certain labels (output) y from the label space Y . Below, we assume classification problems where Y is a finite set ($Y = \{1, 2, \dots, L\}$), however our approach is generic and can handle any output label space that other deep feed-

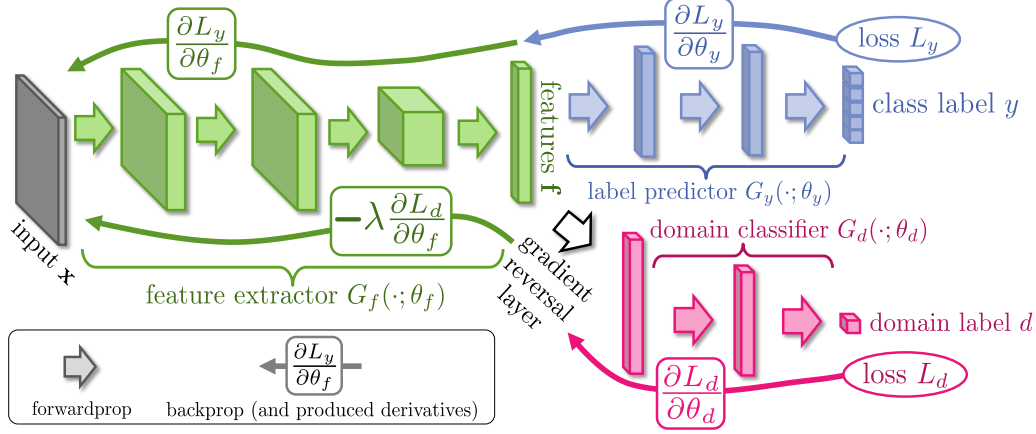


Figure 1. The **proposed architecture** includes a deep *feature extractor* (green) and a deep *label predictor* (blue), which together form a standard feed-forward architecture. Unsupervised domain adaptation is achieved by adding a *domain classifier* (red) connected to the feature extractor via a *gradient reversal layer* that multiplies the gradient by a certain negative constant during the backpropagation-based training. Otherwise, the training proceeds in a standard way and minimizes the label prediction loss (for source examples) and the domain classification loss (for all samples). Gradient reversal ensures that the feature distributions over the two domains are made similar (as indistinguishable as possible for the domain classifier), thus resulting in the domain-invariant features.

forward models can handle. We further assume that there exist two distributions $\mathcal{S}(x, y)$ and $\mathcal{T}(x, y)$ on $X \otimes Y$, which will be referred to as the source distribution and the target distribution (or the source domain and the target domain). Both distributions are assumed complex and unknown, and furthermore similar but different (in other words, $\mathcal{S}$ is “shifted” from $\mathcal{T}$ by some *domain shift*).

Our ultimate goal is to be able to predict labels y given the input x for the target distribution. At training time, we have an access to a large set of training samples $\{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ from both the source and the target domains distributed according to the marginal distributions $\mathcal{S}(\mathbf{x})$ and $\mathcal{T}(\mathbf{x})$. We denote with d_i the binary variable (*domain label*) for the i -th example, which indicates whether x_i come from the source distribution ($\mathbf{x}_i \sim \mathcal{S}(\mathbf{x})$ if $d_i=0$) or from the target distribution ($\mathbf{x}_i \sim \mathcal{T}(\mathbf{x})$ if $d_i=1$). For the examples from the source distribution ($d_i=0$) the corresponding labels $y_i \in Y$ are known at training time. For the examples from the target domains, we do not know the labels at training time, and we want to predict such labels at test time.

We now define a deep feed-forward architecture that for each input $\mathbf{x}$ predicts its label $y \in Y$ and its domain label $d \in \{0, 1\}$. We decompose such mapping into three parts. We assume that the input $\mathbf{x}$ is first mapped by a mapping G_f (a *feature extractor*) to a D -dimensional feature vector $\mathbf{f} \in \mathbb{R}^D$. The feature mapping may also include several feed-forward layers and we denote the vector of parameters of all layers in this mapping as θ_f , i.e. $\mathbf{f} = G_f(\mathbf{x}; \theta_f)$. Then, the feature vector $\mathbf{f}$ is mapped by a mapping G_y (*label predictor*) to the label y , and we denote the parameters of this mapping with θ_y . Finally, the same feature vector $\mathbf{f}$ is mapped to the domain label d by a mapping G_d (*domain*

classifier) with the parameters θ_d (Figure 1).

During the learning stage, we aim to minimize the label prediction loss on the annotated part (i.e. the source part) of the training set, and the parameters of both the feature extractor and the label predictor are thus optimized in order to minimize the empirical loss for the source domain samples. This ensures the discriminativeness of the features $\mathbf{f}$ and the overall good prediction performance of the combination of the feature extractor and the label predictor on the source domain.

At the same time, we want to make the features $\mathbf{f}$ domain-invariant. That is, we want to make the distributions $\mathcal{S}(\mathbf{f}) = \{G_f(\mathbf{x}; \theta_f) | \mathbf{x} \sim \mathcal{S}(\mathbf{x})\}$ and $\mathcal{T}(\mathbf{f}) = \{G_f(\mathbf{x}; \theta_f) | \mathbf{x} \sim \mathcal{T}(\mathbf{x})\}$ to be similar. Under the *covariate shift* assumption, this would make the label prediction accuracy on the target domain to be the same as on the source domain (Shimodaira, 2000). Measuring the dissimilarity of the distributions $\mathcal{S}(\mathbf{f})$ and $\mathcal{T}(\mathbf{f})$ is however non-trivial, given that $\mathbf{f}$ is high-dimensional, and that the distributions themselves are constantly changing as learning progresses. One way to estimate the dissimilarity is to look at the loss of the domain classifier G_d , provided that the parameters θ_d of the domain classifier have been trained to discriminate between the two feature distributions in an optimal way.

This observation leads to our idea. At training time, in order to obtain domain-invariant features, we seek the parameters θ_f of the feature mapping that *maximize* the loss of the domain classifier (by making the two feature distributions as similar as possible), while simultaneously seeking the parameters θ_d of the domain classifier that *minimize* the loss of the domain classifier. In addition, we seek to minimize the loss of the label predictor.

More formally, we consider the functional:

$$\begin{aligned} E(\theta_f, \theta_y, \theta_d) &= \sum_{\substack{i=1..N \\ d_i=0}} L_y(G_y(G_f(\mathbf{x}_i; \theta_f); \theta_y), y_i) - \\ &\quad \lambda \sum_{i=1..N} L_d(G_d(G_f(\mathbf{x}_i; \theta_f); \theta_d), y_i) = \\ &= \sum_{\substack{i=1..N \\ d_i=0}} L_y^i(\theta_f, \theta_y) - \lambda \sum_{i=1..N} L_d^i(\theta_f, \theta_d) \end{aligned} \quad (1)$$

Here, $L_y(\cdot, \cdot)$ is the loss for label prediction (e.g. multinomial), $L_d(\cdot, \cdot)$ is the loss for the domain classification (e.g. logistic), while L_y^i and L_d^i denote the corresponding loss functions evaluated at the i -th training example.

Based on our idea, we are seeking the parameters $\hat{\theta}_f, \hat{\theta}_y, \hat{\theta}_d$ that deliver a saddle point of the functional (1):

$$(\hat{\theta}_f, \hat{\theta}_y) = \arg \min_{\theta_f, \theta_y} E(\theta_f, \theta_y, \hat{\theta}_d) \quad (2)$$

$$\hat{\theta}_d = \arg \max_{\theta_d} E(\hat{\theta}_f, \hat{\theta}_y, \theta_d). \quad (3)$$

At the saddle point, the parameters θ_d of the domain classifier θ_d minimize the domain classification loss (since it enters into (1) with the minus sign) while the parameters θ_y of the label predictor minimize the label prediction loss. The feature mapping parameters θ_f minimize the label prediction loss (i.e. the features are discriminative), while maximizing the domain classification loss (i.e. the features are domain-invariant). The parameter λ controls the trade-off between the two objectives that shape the features during learning.

Below, we demonstrate that standard stochastic gradient solvers (SGD) can be adapted for the search of the saddle point (2)-(3).

3.2. Optimization with backpropagation

A saddle point (2)-(3) can be found as a stationary point of the following stochastic updates:

$$\theta_f \leftarrow \theta_f - \mu \left(\frac{\partial L_y^i}{\partial \theta_f} - \lambda \frac{\partial L_d^i}{\partial \theta_f} \right) \quad (4)$$

$$\theta_y \leftarrow \theta_y - \mu \frac{\partial L_y^i}{\partial \theta_y} \quad (5)$$

$$\theta_d \leftarrow \theta_d - \mu \frac{\partial L_d^i}{\partial \theta_d} \quad (6)$$

where μ is the learning rate (which can vary over time).

The updates (4)-(6) are very similar to stochastic gradient descent (SGD) updates for a feed-forward deep model that comprises feature extractor fed into the label predictor and into the domain classifier. The difference is the $-\lambda$ factor in (4) (the difference is important, as without such factor,

stochastic gradient descent would try to make features dissimilar across domains in order to minimize the domain classification loss). Although direct implementation of (4)-(6) as SGD is not possible, it is highly desirable to reduce the updates (4)-(6) to some form of SGD, since SGD (and its variants) is the main learning algorithm implemented in most packages for deep learning.

Fortunately, such reduction can be accomplished by introducing a special **gradient reversal layer** (GRL) defined as follows. The gradient reversal layer has no parameters associated with it (apart from the meta-parameter λ , which is not updated by backpropagation). During the forward propagation, GRL acts as an identity transform. During the backpropagation though, GRL takes the gradient from the subsequent level, multiplies it by $-\lambda$ and passes it to the preceding layer. Implementing such layer using existing object-oriented packages for deep learning is simple, as defining procedures for forwardprop (identity transform), backprop (multiplying by a constant), and parameter update (nothing) is trivial.

The GRL as defined above is inserted between the feature extractor and the domain classifier, resulting in the architecture depicted in Figure 1. As the backpropagation process passes through the GRL, the partial derivatives of the loss that is downstream the GRL (i.e. L_d) w.r.t. the layer parameters that are upstream the GRL (i.e. θ_f) get multiplied by $-\lambda$, i.e. $\frac{\partial L_d}{\partial \theta_f}$ is effectively replaced with $-\lambda \frac{\partial L_d}{\partial \theta_f}$. Therefore, running SGD in the resulting model implements the updates (4)-(6) and converges to a saddle point of (1). Mathematically, we can formally treat the gradient reversal layer as a “pseudo-function” $R_\lambda(\mathbf{x})$ defined by two (incompatible) equations describing its forward- and backpropagation behaviour:

$$R_\lambda(\mathbf{x}) = \mathbf{x} \quad (7)$$

$$\frac{dR_\lambda}{d\mathbf{x}} = -\lambda \mathbf{I} \quad (8)$$

where $\mathbf{I}$ is an identity matrix. We can then define the objective “pseudo-function” of $(\theta_f, \theta_y, \theta_d)$ that is being optimized by the stochastic gradient descent within our method:

$$\begin{aligned} \tilde{E}(\theta_f, \theta_y, \theta_d) &= \sum_{\substack{i=1..N \\ d_i=0}} L_y(G_y(G_f(\mathbf{x}_i; \theta_f); \theta_y), y_i) + \\ &\quad \sum_{i=1..N} L_d(G_d(R_\lambda(G_f(\mathbf{x}_i; \theta_f)); \theta_d), y_i) \end{aligned} \quad (9)$$

Running updates (4)-(6) can then be implemented as doing SGD for (9) and leads to the emergence of features that are domain-invariant and discriminative at the same time. After the learning, the label predictor $y(\mathbf{x}) = G_y(G_f(\mathbf{x}; \theta_f); \theta_y)$ can be used to predict labels for samples from the target domain (as well as from the source domain).

The simple learning procedure outlined above can be re-derived/generalized along the lines suggested in (Goodfellow et al., 2014) (see Appendix A).

3.3. Relation to $\mathcal{H}\Delta\mathcal{H}$ -distance

In this section we give a brief analysis of our method in terms of $\mathcal{H}\Delta\mathcal{H}$ -distance (Ben-David et al., 2010; Cortes & Mohri, 2011) which is widely used in the theory of non-conservative domain adaptation. Formally,

$$d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{S}, \mathcal{T}) = 2 \sup_{h_1, h_2 \in \mathcal{H}} |P_{\mathbf{f} \sim \mathcal{S}}[h_1(\mathbf{f}) \neq h_2(\mathbf{f})] - P_{\mathbf{f} \sim \mathcal{T}}[h_1(\mathbf{f}) \neq h_2(\mathbf{f})]| \quad (10)$$

defines a discrepancy distance between two distributions $\mathcal{S}$ and $\mathcal{T}$ w.r.t. a hypothesis set $\mathcal{H}$. Using this notion one can obtain a probabilistic bound (Ben-David et al., 2010) on the performance $\varepsilon_{\mathcal{T}}(h)$ of some classifier h from $\mathcal{T}$ evaluated on the target domain given its performance $\varepsilon_{\mathcal{S}}(h)$ on the source domain:

$$\varepsilon_{\mathcal{T}}(h) \leq \varepsilon_{\mathcal{S}}(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(\mathcal{S}, \mathcal{T}) + C, \quad (11)$$

where $\mathcal{S}$ and $\mathcal{T}$ are source and target distributions respectively, and C does not depend on particular h .

Consider fixed $\mathcal{S}$ and $\mathcal{T}$ over the representation space produced by the feature extractor G_f and a family of label predictors $\mathcal{H}_p$. We assume that the family of domain classifiers $\mathcal{H}_d$ is rich enough to contain the symmetric difference hypothesis set of $\mathcal{H}_p$:

$$\mathcal{H}_p \Delta \mathcal{H}_p = \{h \mid h = h_1 \oplus h_2, h_1, h_2 \in \mathcal{H}_p\}. \quad (12)$$

It is not an unrealistic assumption as we have a freedom to pick $\mathcal{H}_d$ whichever we want. For example, we can set the architecture of the domain discriminator to be the layer-by-layer concatenation of two replicas of the label predictor followed by a two layer non-linear perceptron aimed to learn the XOR-function. Given the assumption holds, one can easily show that training the G_d is closely related to the estimation of $d_{\mathcal{H}_p \Delta \mathcal{H}_p}(\mathcal{S}, \mathcal{T})$. Indeed,

$$\begin{aligned} d_{\mathcal{H}_p \Delta \mathcal{H}_p}(\mathcal{S}, \mathcal{T}) &= \\ &= 2 \sup_{h \in \mathcal{H}_p \Delta \mathcal{H}_p} |P_{\mathbf{f} \sim \mathcal{S}}[h(\mathbf{f}) = 1] - P_{\mathbf{f} \sim \mathcal{T}}[h(\mathbf{f}) = 1]| \leq \\ &\leq 2 \sup_{h \in \mathcal{H}_d} |P_{\mathbf{f} \sim \mathcal{S}}[h(\mathbf{f}) = 1] - P_{\mathbf{f} \sim \mathcal{T}}[h(\mathbf{f}) = 1]| = \\ &= 2 \sup_{h \in \mathcal{H}_d} |1 - \alpha(h)| = 2 \sup_{h \in \mathcal{H}_d} [\alpha(h) - 1] \end{aligned} \quad (13)$$

where $\alpha(h) = P_{\mathbf{f} \sim \mathcal{S}}[h(\mathbf{f}) = 0] + P_{\mathbf{f} \sim \mathcal{T}}[h(\mathbf{f}) = 1]$ is maximized by the optimal G_d .

Thus, optimal discriminator gives the upper bound for $d_{\mathcal{H}_p \Delta \mathcal{H}_p}(\mathcal{S}, \mathcal{T})$. At the same time, backpropagation of the reversed gradient changes the representation space

so that $\alpha(G_d)$ becomes smaller effectively reducing $d_{\mathcal{H}_p \Delta \mathcal{H}_p}(\mathcal{S}, \mathcal{T})$ and leading to the better approximation of $\varepsilon_{\mathcal{T}}(G_y)$ by $\varepsilon_{\mathcal{S}}(G_y)$.

4. Experiments

We perform extensive evaluation of the proposed approach on a number of popular image datasets and their modifications. These include large-scale datasets of small images popular with deep learning methods, and the OFFICE datasets (Saenko et al., 2010), which are a *de facto* standard for domain adaptation in computer vision, but have much fewer images.

Baselines. For the bulk of experiments the following baselines are evaluated. The **source-only** model is trained without consideration for target-domain data (no domain classifier branch included into the network). The **train-on-target** model is trained on the target domain with class labels revealed. This model serves as an upper bound on DA methods, assuming that target data are abundant and the shift between the domains is considerable.

In addition, we compare our approach against the recently proposed unsupervised DA method based on **subspace alignment (SA)** (Fernando et al., 2013), which is simple to setup and test on new datasets, but has also been shown to perform very well in experimental comparisons with other “shallow” DA methods. To boost the performance of this baseline, we pick its most important free parameter (the number of principal components) from the range $\{2, \dots, 60\}$, so that the test performance on the target domain is maximized. To apply SA in our setting, we train a source-only model and then consider the activations of the last hidden layer in the label predictor (before the final linear classifier) as descriptors/features, and learn the mapping between the source and the target domains (Fernando et al., 2013).

Since the SA baseline requires to train a new classifier after adapting the features, and in order to put all the compared settings on an equal footing, we retrain the last layer of the label predictor using a standard linear SVM (Fan et al., 2008) for all four considered methods (including ours; the performance on the target domain remains approximately the same after the retraining).

For the OFFICE dataset (Saenko et al., 2010), we directly compare the performance of our full network (feature extractor and label predictor) against recent DA approaches using previously published results.

CNN architectures. In general, we compose feature extractor from two or three convolutional layers, picking their exact configurations from previous works. We give the exact architectures in Appendix B.

For the domain adaptator we stick to the three fully connected layers ($x \rightarrow 1024 \rightarrow 1024 \rightarrow 2$), except for MNIST where we used a simpler ($x \rightarrow 100 \rightarrow 2$) ar-



Figure 2. Examples of domain pairs used in the experiments. See Section 4.1 for details.

METHOD	SOURCE	MNIST	SYN NUMBERS	SVHN	SYN SIGNS
	TARGET	MNIST-M	SVHN	MNIST	GTSRB
SOURCE ONLY		.5749	.8665	.5919	.7400
SA (FERNANDO ET AL., 2013)		.6078 (7.9%)	.8672 (1.3%)	.6157 (5.9%)	.7635 (9.1%)
PROPOSED APPROACH		.8149 (57.9%)	.9048 (66.1%)	.7107 (29.3%)	.8866 (56.7%)
TRAIN ON TARGET		.9891	.9244	.9951	.9987

Table 1. Classification accuracies for digit image classifications for different source and target domains. MNIST-M corresponds to difference-blended digits over non-uniform background. The first row corresponds to the lower performance bound (i.e. if no adaptation is performed). The last row corresponds to training on the target domain data with known class labels (upper bound on the DA performance). For each of the two DA methods (ours and (Fernando et al., 2013)) we show how much of the gap between the lower and the upper bounds was covered (in brackets). For all five cases, our approach outperforms (Fernando et al., 2013) considerably, and covers a big portion of the gap.

chitecture to speed up the experiments.

For loss functions, we set L_y and L_d to be the logistic regression loss and the binomial cross-entropy respectively.

CNN training procedure. The model is trained on 128-sized batches. Images are preprocessed by the mean subtraction. A half of each batch is populated by the samples from the source domain (with known labels), the rest is comprised of the target domain (with unknown labels). In order to suppress noisy signal from the domain classifier at the early stages of the training procedure instead of fixing the adaptation factor λ , we gradually change it from 0 to 1 using the following schedule:

$$\lambda_p = \frac{2}{1 + \exp(-\gamma \cdot p)} - 1, \quad (14)$$

where γ was set to 10 in all experiments (the schedule was not optimized/tweaked). Further details on the CNN training can be found in Appendix C.

Visualizations. We use t-SNE (van der Maaten, 2013) projection to visualize feature distributions at different points of the network, while color-coding the domains (Figure 3). We observe strong correspondence between the success of the adaptation in terms of the classification accuracy for the target domain, and the overlap between the domain distributions in such visualizations.

Choosing meta-parameters. In general, good unsupervised DA methods should provide ways to set meta-parameters (such as λ , the learning rate, the momentum

rate, the network architecture for our method) in an unsupervised way, i.e. without referring to labeled data in the target domain. In our method, one can assess the performance of the whole system (and the effect of changing hyper-parameters) by observing the test error on the source domain and the domain classifier error. In general, we observed a good correspondence between the success of adaptation and these errors (adaptation is more successful when the source domain test error is low, while the domain classifier error is high). In addition, the layer, where the domain adaptor is attached can be picked by computing difference between means as suggested in (Tzeng et al., 2014).

4.1. Results

We now discuss the experimental settings and the results. In each case, we train on the source dataset and test on a different target domain dataset, with considerable shifts between domains (see Figure 2). The results are summarized in Table 1 and Table 2.

MNIST $\rightarrow$ MNIST-M. Our first experiment deals with the MNIST dataset (LeCun et al., 1998) (source). In order to obtain the target domain (MNIST-M) we blend digits from the original set over patches randomly extracted from color photos from BSDS500 (Arbelaez et al., 2011). This operation is formally defined for two images I^1, I^2 as $I_{ijk}^{out} = |I_{ijk}^1 - I_{ijk}^2|$, where i, j are the coordinates of a pixel and k is a channel index. In other words, an output sample is produced by taking a patch from a photo and in-

METHOD	SOURCE	AMAZON	DSLR	WEBCAM
	TARGET	WEBCAM	WEBCAM	DSLR
GFK(PLS, PCA) (GONG ET AL., 2012)		.464 $\pm$.005	.613 $\pm$.004	.663 $\pm$.004
SA (FERNANDO ET AL., 2013)		.450	.648	.699
DA-NBNN (TOMMASI & CAPUTO, 2013)		.528 $\pm$.037	.766 $\pm$.017	.762 $\pm$.025
DLID (S. CHOPRA & GOPALAN, 2013)		.519	.782	.899
DECAF ₆ SOURCE ONLY (DONAHUE ET AL., 2014)		.522 $\pm$.017	.915 $\pm$.015	–
DANN (GHIFARY ET AL., 2014)		.536 $\pm$.002	.712 $\pm$.000	.835 $\pm$.000
DDC (TZENG ET AL., 2014)		.594 $\pm$.008	.925 $\pm$.003	.917 $\pm$.008
PROPOSED APPROACH		.673 $\pm$.017	.940 $\pm$.008	.937 $\pm$.010

Table 2. Accuracy evaluation of different DA approaches on the standard OFFICE (Saenko et al., 2010) dataset. Our method (last row) outperforms competitors setting the new state-of-the-art.

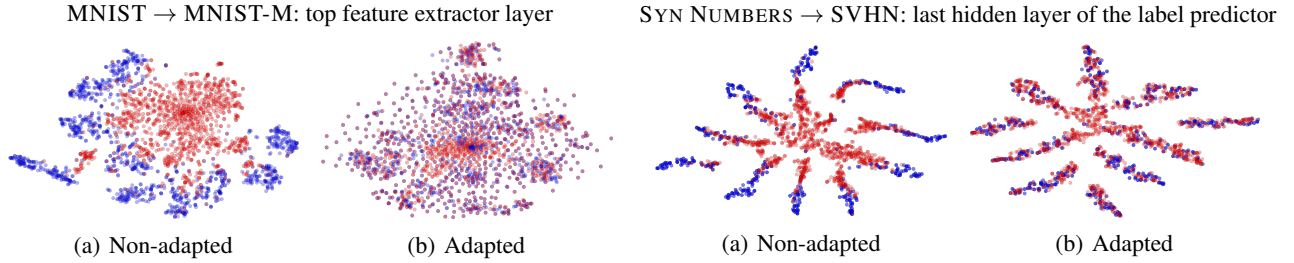


Figure 3. The effect of adaptation on the distribution of the extracted features (best viewed in color). The figure shows t-SNE (van der Maaten, 2013) visualizations of the CNN’s activations (a) in case when no adaptation was performed and (b) in case when our adaptation procedure was incorporated into training. Blue points correspond to the source domain examples, while red ones correspond to the target domain. In all cases, the adaptation in our method makes the two distributions of features much closer.

verting its pixels at positions corresponding to the pixels of a digit. For a human the classification task becomes only slightly harder compared to the original dataset (the digits are still clearly distinguishable) whereas for a CNN trained on MNIST this domain is quite distinct, as the background and the strokes are no longer constant. Consequently, the source-only model performs poorly. Our approach succeeded at aligning feature distributions (Figure 3), which led to successful adaptation results (considering that the adaptation is unsupervised). At the same time, the improvement over source-only model achieved by subspace alignment (SA) (Fernando et al., 2013) is quite modest, thus highlighting the difficulty of the adaptation task.

Synthetic numbers → SVHN. To address a common scenario of training on synthetic data and testing on real data, we use Street-View House Number dataset SVHN (Netzer et al., 2011) as the target domain and synthetic digits as the source. The latter (SYN NUMBERS) consists of 500,000 images generated by ourselves from Windows fonts by varying the text (that includes different one-, two-, and three-digit numbers), positioning, orientation, background and stroke colors, and the amount of blur. The degrees of

variation were chosen manually to simulate SVHN, however the two datasets are still rather distinct, the biggest difference being the structured clutter in the background of SVHN images.

The proposed backpropagation-based technique works well covering two thirds of the gap between training with source data only and training on target domain data with known target labels. In contrast, SA (Fernando et al., 2013) does not result in any significant improvement in the classification accuracy, thus highlighting that the adaptation task is even more challenging than in the case of the MNIST experiment.

MNIST ↔ SVHN. In this experiment, we further increase the gap between distributions, and test on MNIST and SVHN, which are significantly different in appearance. Training on SVHN even without adaptation is challenging — classification error stays high during the first 150 epochs. In order to avoid ending up in a poor local minimum we, therefore, do not use learning rate annealing here. Obviously, the two directions (MNIST → SVHN and SVHN → MNIST) are not equally difficult. As SVHN is more diverse, a model trained on SVHN is ex-

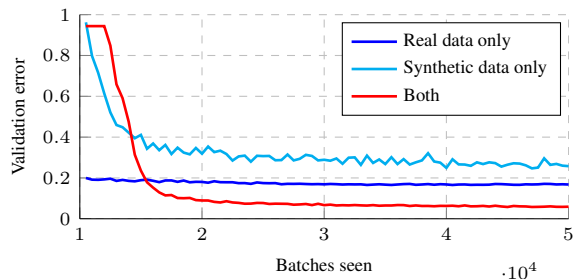


Figure 4. Semi-supervised domain adaptation for the traffic signs. As labeled target domain data are shown to the method, it achieves significantly lower error than the model trained on target domain data only or on source domain data only.

pected to be more generic and to perform reasonably on the MNIST dataset. This, indeed, turns out to be the case and is supported by the appearance of the feature distributions. We observe a quite strong separation between the domains when we feed them into the CNN trained solely on MNIST, whereas for the SVHN-trained network the features are much more intermixed. This difference probably explains why our method succeeded in improving the performance by adaptation in the SVHN $\rightarrow$ MNIST scenario (see Table 1) but not in the opposite direction (SA is not able to perform adaptation in this case either). Unsupervised adaptation from MNIST to SVHN gives a failure example for our approach (we are unaware of any unsupervised DA methods capable of performing such adaptation).

Synthetic Signs $\rightarrow$ GTSRB. Overall, this setting is similar to the SYN NUMBERS $\rightarrow$ SVHN experiment, except the distribution of the features is more complex due to the significantly larger number of classes (43 instead of 10). For the source domain we obtained 100,000 synthetic images (which we call SYN SIGNS) simulating various photoshooting conditions. Once again, our method achieves a sensible increase in performance once again proving its suitability for the synthetic-to-real data adaptation.

As an additional experiment, we also evaluate the proposed algorithm for semi-supervised domain adaptation, i.e. when one is additionally provided with a small amount of labeled target data. For that purpose we split GTSRB into the train set (1280 random samples with labels) and the validation set (the rest of the dataset). The validation part is used solely for the evaluation and does not participate in the adaptation. The training procedure changes slightly as the label predictor is now exposed to the target data. Figure 4 shows the change of the validation error throughout the training. While the graph clearly suggests that our method can be used in the semi-supervised setting, thorough verification of semi-supervised setting is left for future work.

Office dataset. We finally evaluate our method on OFFICE dataset, which is a collection of three distinct do-

main: AMAZON, DSLR, and WEBCAM. Unlike previously discussed datasets, OFFICE is rather small-scale with only 2817 labeled images spread across 31 different categories in the largest domain. The amount of available data is crucial for a successful training of a deep model, hence we opted for the fine-tuning of the CNN pre-trained on the ImageNet (Jia et al., 2014) as it is done in some recent DA works (Donahue et al., 2014; Tzeng et al., 2014; Hoffman et al., 2013). We make our approach more comparable with (Tzeng et al., 2014) by using exactly the same network architecture replacing domain mean-based regularization with the domain classifier.

Following most previous works, we evaluate our method using 5 random splits for each of the 3 transfer tasks commonly used for evaluation. Our training protocol is close to (Tzeng et al., 2014; Saenko et al., 2010; Gong et al., 2012) as we use the same number of labeled source-domain images per category. Unlike those works and similarly to e.g. DLID (S. Chopra & Gopalan, 2013) we use the whole unlabeled target domain (as the premise of our method is the abundance of unlabeled data in the target domain). Under this transductive setting, our method is able to improve previously-reported state-of-the-art accuracy for unsupervised adaptation very considerably (Table 2), especially in the most challenging AMAZON $\rightarrow$ WEBCAM scenario (the two domains with the largest domain shift).

5. Discussion

We have proposed a new approach to unsupervised domain adaptation of deep feed-forward architectures, which allows large-scale training based on large amount of annotated data in the source domain and large amount of unannotated data in the target domain. Similarly to many previous shallow and deep DA techniques, the adaptation is achieved through aligning the distributions of features across the two domains. However, unlike previous approaches, the alignment is accomplished through standard backpropagation training. The approach is therefore rather scalable, and can be implemented using any deep learning package. To this end we plan to release the source code for the Gradient Reversal layer along with the usage examples as an extension to Caffe (Jia et al., 2014).

Further evaluation on larger-scale tasks and in semi-supervised settings constitutes future work. It is also interesting whether the approach can benefit from a good initialization of the feature extractor. For this, a natural choice would be to use deep autoencoder/deconvolution network trained on both domains (or on the target domain) in the same vein as (Glorot et al., 2011; S. Chopra & Gopalan, 2013), effectively using (Glorot et al., 2011; S. Chopra & Gopalan, 2013) as an initialization to our method.

Appendix A. An alternative optimization approach

There exists an alternative construction (inspired by (Goodfellow et al., 2014)) that leads to the same updates (4)-(6). Rather than using the gradient reversal layer, the construction introduces two different loss functions for the domain classifier. Minimization of the first domain loss (L_{d+}) should lead to a better domain discrimination, while the second domain loss (L_{d-}) is minimized when the domains are distinct. Stochastic updates for θ_f and θ_d are then defined as:

$$\begin{aligned}\theta_f &\leftarrow \theta_f - \mu \left(\frac{\partial L_y^i}{\partial \theta_f} + \frac{\partial L_{d-}^i}{\partial \theta_f} \right) \\ \theta_d &\leftarrow \theta_d - \mu \frac{\partial L_{d+}^i}{\partial \theta_d},\end{aligned}$$

Thus, different parameters participate in the optimization of different losses

In this framework, the gradient reversal layer constitutes a special case, corresponding to the pair of domain losses ($L_d, -\lambda L_d$). However, other pairs of loss functions can be used. One example would be the binomial cross-entropy (Goodfellow et al., 2014):

$$L_{d+}(q, d) = \sum_{i=1..N} d_i \log(q_i) + (1 - d_i) \log(1 - q_i),$$

where d indicates domain indices and q is an output of the predictor. In that case “adversarial” loss is easily obtained by swapping domain labels, i.e. $L_{d-}(q, d) = L_{d+}(q, 1-d)$. This particular pair has a potential advantage of producing stronger gradients at early learning stages if the domains are quite dissimilar. In our experiments, however, we did not observe any significant improvement resulting from this choice of losses.

Appendix B. CNN architectures

Four different architectures were used in our experiments (first three are shown in Figure 5):

- A smaller one (a) if the source domain is MNIST. This architecture was inspired by the classical LeNet-5 (Lecun et al., 1998).
- (b) for the experiments involving SVHN dataset. This one is adopted from (Srivastava et al., 2014).
- (c) in the SYN SINGS $\rightarrow$ GTSRB setting. We used the single-CNN baseline from (Cireřan et al., 2012) as our starting point.
- Finally, we use pre-trained AlexNet from the Caffe-package (Jia et al., 2014) for the OFFICE domains. Adaptation architecture is identical to (Tzeng et al., 2014): 2-layer domain classifier ($x \rightarrow 1024 \rightarrow$

$1024 \rightarrow 2$) is attached to the 256-dimensional bottleneck of fc7.

The domain classifier branch in all cases is somewhat arbitrary (better adaptation performance might be attained if this part of the architecture is tuned).

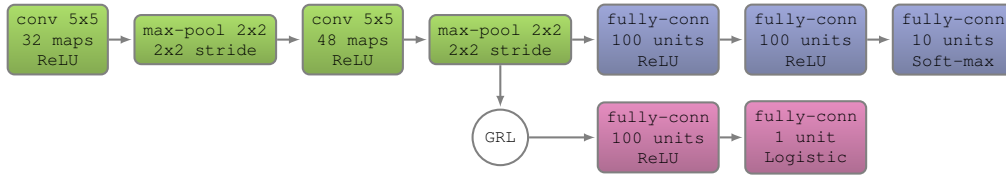
Appendix C. Training procedure

We use stochastic gradient descent with 0.9 momentum and the learning rate annealing described by the following formula:

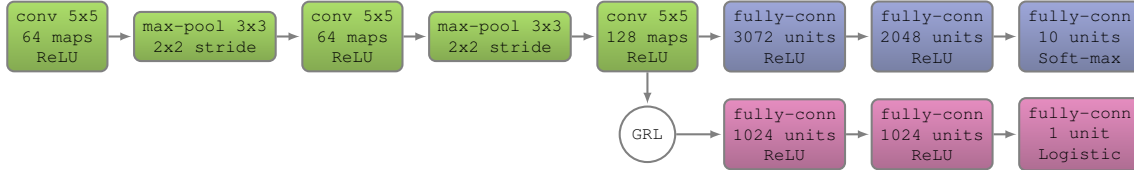
$$\mu_p = \frac{\mu_0}{(1 + \alpha \cdot p)^\beta},$$

where p is the training progress linearly changing from 0 to 1, $\mu_0 = 0.01$, $\alpha = 10$ and $\beta = 0.75$ (the schedule was optimized to promote convergence and low error on the *source* domain).

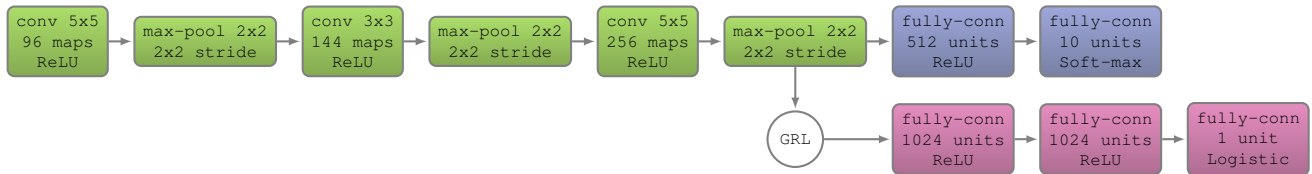
Following (Srivastava et al., 2014) we also use dropout and ℓ_2 -norm restriction when we train the SVHN architecture.



(a) MNIST architecture



(b) SVHN architecture



(c) GTSRB architecture

Figure 5. CNN architectures used in the experiments. Boxes correspond to transformations applied to the data. Color-coding is the same as in Figure 1.

References

- Arbelaez, Pablo, Maire, Michael, Fowlkes, Charless, and Malik, Jitendra. Contour detection and hierarchical image segmentation. *PAMI*, 33, 2011.
- Babenko, Artem, Slesarev, Anton, Chigorin, Alexander, and Lempitsky, Victor S. Neural codes for image retrieval. In *ECCV*, pp. 584–599, 2014.
- Baktashmotlagh, Mahsa, Harandi, Mehrtash Tafazzoli, Lovell, Brian C., and Salzmann, Mathieu. Unsupervised domain adaptation by domain invariant projection. In *ICCV*, pp. 769–776, 2013.
- Ben-David, Shai, Blitzer, John, Crammer, Koby, Kulesza, Alex, Pereira, Fernando, and Vaughan, Jennifer Wortman. A theory of learning from different domains. *JMLR*, 79, 2010.
- Borgwardt, Karsten M., Gretton, Arthur, Rasch, Malte J., Kriegel, Hans-Peter, Schölkopf, Bernhard, and Smola, Alexander J. Integrating structured biological data by kernel maximum mean discrepancy. In *ISMB*, pp. 49–57, 2006.
- Cireşan, Dan, Meier, Ueli, Masci, Jonathan, and Schmidhuber, Jürgen. Multi-column deep neural network for traffic sign classification. *Neural Networks*, (32):333–338, 2012.
- Cortes, Corinna and Mohri, Mehryar. Domain adaptation in regression. In *Algorithmic Learning Theory*, 2011.
- Donahue, Jeff, Jia, Yangqing, Vinyals, Oriol, Hoffman, Judy, Zhang, Ning, Tzeng, Eric, and Darrell, Trevor. Decaf: A deep convolutional activation feature for generic visual recognition, 2014.
- Fan, Rong-En, Chang, Kai-Wei, Hsieh, Cho-Jui, Wang, Xiang-Rui, and Lin, Chih-Jen. LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research*, 9:1871–1874, 2008.
- Fernando, Basura, Habrard, Amaury, Sebban, Marc, and Tuytelaars, Tinne. Unsupervised visual domain adaptation using subspace alignment. In *ICCV*, 2013.
- Ghifary, Muhammad, Kleijn, W Bastiaan, and Zhang, Mengjie. Domain adaptive neural networks for object recognition. In *PRICAI 2014: Trends in Artificial Intelligence*. 2014.
- Glorot, Xavier, Bordes, Antoine, and Bengio, Yoshua. Domain adaptation for large-scale sentiment classification: A deep learning approach. In *ICML*, pp. 513–520, 2011.
- Gong, Boqing, Shi, Yuan, Sha, Fei, and Grauman, Kristen. Geodesic flow kernel for unsupervised domain adaptation. In *CVPR*, pp. 2066–2073, 2012.

- Gong, Boqing, Grauman, Kristen, and Sha, Fei. Connecting the dots with landmarks: Discriminatively learning domain-invariant features for unsupervised domain adaptation. In *ICML*, pp. 222–230, 2013.
- Goodfellow, Ian, Pouget-Abadie, Jean, Mirza, Mehdi, Xu, Bing, Warde-Farley, David, Ozair, Sherjil, Courville, Aaron, and Bengio, Yoshua. Generative adversarial nets. In *NIPS*, 2014.
- Gopalan, Raghuraman, Li, Ruonan, and Chellappa, Rama. Domain adaptation for object recognition: An unsupervised approach. In *ICCV*, pp. 999–1006, 2011.
- Hoffman, Judy, Tzeng, Eric, Donahue, Jeff, Jia, Yangqing, Saenko, Kate, and Darrell, Trevor. One-shot adaptation of supervised deep convolutional models. *CoRR*, abs/1312.6204, 2013.
- Huang, Jiayuan, Smola, Alexander J., Gretton, Arthur, Borgwardt, Karsten M., and Schölkopf, Bernhard. Correcting sample selection bias by unlabeled data. In *NIPS*, pp. 601–608, 2006.
- Jia, Yangqing, Shelhamer, Evan, Donahue, Jeff, Karayev, Sergey, Long, Jonathan, Girshick, Ross, Guadarrama, Sergio, and Darrell, Trevor. Caffe: Convolutional architecture for fast feature embedding. *CoRR*, abs/1408.5093, 2014.
- LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, November 1998.
- Liebelt, Joerg and Schmid, Cordelia. Multi-view object class detection with a 3d geometric model. In *CVPR*, 2010.
- Netzer, Yuval, Wang, Tao, Coates, Adam, Bissacco, Alessandro, Wu, Bo, and Ng, Andrew Y. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011.
- Oquab, M., Bottou, L., Laptev, I., and Sivic, J. Learning and transferring mid-level image representations using convolutional neural networks. In *CVPR*, 2014.
- Pan, Sinno Jialin, Tsang, Ivor W., Kwok, James T., and Yang, Qiang. Domain adaptation via transfer component analysis. *IEEE Transactions on Neural Networks*, 22(2): 199–210, 2011.
- S. Chopra, S. Balakrishnan and Gopalan, R. Dlid: Deep learning for domain adaptation by interpolating between domains. In *ICML Workshop on Challenges in Representation Learning*, 2013.
- Saenko, Kate, Kulis, Brian, Fritz, Mario, and Darrell, Trevor. Adapting visual category models to new domains. In *ECCV*, pp. 213–226. 2010.
- Shimodaira, Hidetoshi. Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90 (2):227–244, October 2000.
- Srivastava, Nitish, Hinton, Geoffrey, Krizhevsky, Alex, Sutskever, Ilya, and Salakhutdinov, Ruslan. Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1): 1929–1958, 2014.
- Stark, Michael, Goesele, Michael, and Schiele, Bernt. Back to the future: Learning shape models from 3d CAD data. In *BMVC*, pp. 1–11, 2010.
- Sun, Baochen and Saenko, Kate. From virtual to reality: Fast adaptation of virtual object detectors to real domains. In *BMVC*, 2014.
- Tommasi, Tatiana and Caputo, Barbara. Frustratingly easy nbnn domain adaptation. In *ICCV*, 2013.
- Tzeng, Eric, Hoffman, Judy, Zhang, Ning, Saenko, Kate, and Darrell, Trevor. Deep domain confusion: Maximizing for domain invariance. *CoRR*, abs/1412.3474, 2014.
- van der Maaten, Laurens. Barnes-hut-sne. *CoRR*, abs/1301.3342, 2013.
- Vázquez, David, López, Antonio Manuel, Marín, Javier, Ponsa, Daniel, and Gomez, David Gerónimo. Virtual and real world adaptation for pedestrian detection. *IEEE Trans. Pattern Anal. Mach. Intell.*, 36(4):797–809, 2014.
- Zeiler, Matthew D. and Fergus, Rob. Visualizing and understanding convolutional networks. *CoRR*, abs/1311.2901, 2013.